

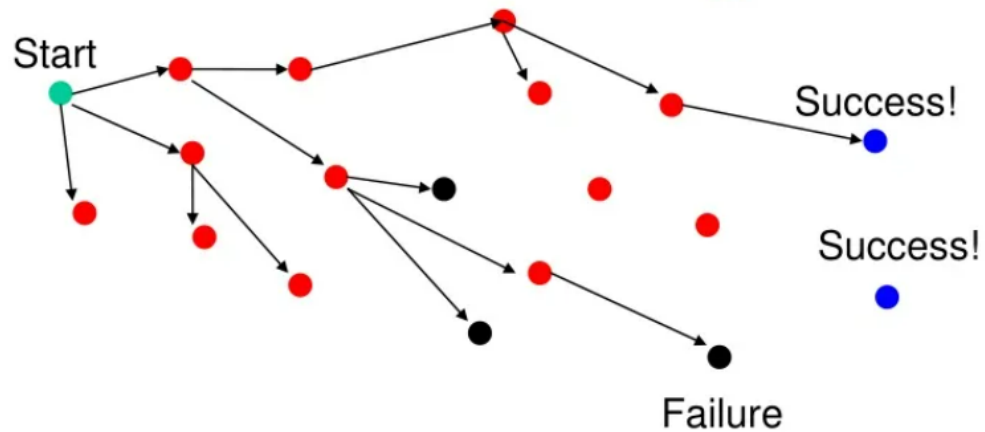
Backtracking

Backtracking

- Backtracking (перебор с возвратом) — это алгоритмическая техника для решения задач, где требуется найти все (или некоторые) комбинации объектов, удовлетворяющих заданным условиям.
- Она основана на последовательном построении кандидатов в решения и откате (возврате) при обнаружении тупиковой ветви.
- Это ключевой метод для задач с множеством вариантов, которые нужно проверить, но где полный перебор неэффективен.

Пример

Backtracking



Как работает Backtracking?

1. Пошаговое построение решения:

На каждом шаге делается выбор (например, размещение ферзя на доске, добавление цифры в комбинацию).

2. Проверка условий:

Если текущий выбор нарушает правила, алгоритм откатывается к предыдущему шагу.

3. Рекурсия:

Процесс повторяется рекурсивно, пока не будет найдено полное решение или все варианты не будут перебраны.

4. Откат (Backtrack):

Если путь зашёл в тупик, алгоритм возвращается на предыдущий шаг и пробует другую ветвь.

Аналогия с лабиринтом

Представьте, что вы ищете выход в лабиринте:

1. Идёте по одному пути.
2. Если упираетесь в стену — возвращаетесь к последней развилке и пробуете другую дорогу.
3. Так вы исследуете все возможные пути, но не тратите время на заведомо тупиковые ветви после отката.

Пример. Перестановки чисел

1. Перестановки чисел

Условие: Дан массив уникальных чисел. Верните все возможные перестановки.

Пример входа: [1,2,3]

Пример выхода: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Пример кода

python

Copy Download

```
def permute(nums):
    result = []
    used = [False] * len(nums) # Отслеживаем использованные элементы

    def backtrack(path):
        if len(path) == len(nums):
            result.append(path.copy())
            return
        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                path.append(nums[i])
                backtrack(path) # Рекурсия
                path.pop()      # Откат
                used[i] = False

    backtrack([])
    return result

print(permute([1, 2, 3]))
```

2. Подмножества

2. Подмножества

Условие: Дан массив уникальных чисел. Верните все возможные подмножества.

Пример входа: [1,2,3]

Пример выхода: [[],[1],[2],[3],[1,2],[1,3],[2,3],[1,2,3]]

3. Путь в лабиринте

3. Путь в лабиринте

Условие: Найдите путь от (0,0) до (N-1,M-1) в матрице с препятствиями (1-стена).

Пример входа:

[[0,0,0],

[1,1,0],

[0,0,0]]

Пример выхода:

Существует путь: Да

4. Генерация скобок

4. Генерация скобок

Условие: Создайте все правильные комбинации из N пар скобок.

Пример входа: N=3

Пример выхода: ['((()))', '(())', '() ()', ') ()', ')) ()']

5. Комбинации сумм

5. Комбинации сумм

Условие: Найдите комбинации чисел из массива, дающие целевую сумму.

Пример входа: [2,3,6,7], target=7

Пример выхода: [[2,2,3],[7]]